開発計画書: DarkPrint Optimizer

ダークモードのスクリーンショットを印刷用に最適化し、インキ・トナー消費を最小限に抑えつつ視認性を最大化する画像処理エンジンの開発計画。

1. プロジェクト概要

近年のダークモード普及に伴い、黒背景の画面を印刷する際のコストと視認性の低下が課題となっている。本プロジェクトでは、コンピュータビジョン技術を用いてUI要素とコンテンツ（写真・アイコン）をインテリジェントに分離し、印刷に最適な「白背景・高コントラスト」な画像を生成する。

2. 技術スタック

- **コア言語:** Python 3.x (プロトタイプ), Rust (製品版エンジン)
- **画像処理:** OpenCV (Open Source Computer Vision Library)
- **領域解析:** k-means クラスタリング, Canny法, ラプラシアンフィルタ
- **配布形態:** CLIツール, ブラウザ拡張機能, 仮想プリンタードライバー

3. 主要アルゴリズム

3.1 背景特定 (k-means)

画像全体から主要な色を抽出し、最大面積を占める暗色を背景色として定義する。これによりアプリごとの「微妙な黒」に柔軟に対応する。

3.2 コンテンツ保護 (Edge Density Analysis)

Cannyエッジ検出と膨張処理（Dilation）を組み合わせ、エッジ密度の高い領域を「保護すべきコンテンツ（写真等）」としてマスク化する。

3.3 文字コントラスト補正 (LUT)

反転後に薄くなったグレー文字を、トーンカーブを用いて引き締める。変換関数：

$$f(x) = 255 \times ((x - threshold) / (255 - threshold))^{\gamma}$$

3.4 UI骨格抽出 (Laplacian)

ボタンや枠線を「塗りつぶし」から「線画」へ変換。ラプラシアンフィルタによりエッジのみを抽出し、インキ消費を抑えつつ構造を維持する。

4. 開発ロードマップ

フェーズ	目標	主要技術
Phase 1	CLIプロトタイプの完成	Python, OpenCV, k-means
Phase 2	ブラウザ拡張版のリリース	WebAssembly, JavaScript (DOM解析)
Phase 3	OSレベル仮想プリンター開発	C++/Rust, GDI/CUPS Filters

5. 評価指標

- **インキ削減率:** 変換前後のCMYK総量の比較。目標：80%以上削減。
- **視認性 (OCR精度):** 変換後のテキスト領域に対するOCR認識率の維持。